Optimization: Making Learning Move

Chapter 4 · Gradient descent, difficult terrain, momentum, adaptive methods, and diagnosis

Dr. A. Belcaid

Table of contents

1 From a model to a landscape **2**

2 Gradient descent: direction and distance **4**
2.1 Practice: execute one update 4
2.2 Learning-rate regimes 5

3 Three gradient regimes **5**

4 Difficult terrain **7**
4.1 Ravines and unequal curvature 7
4.2 Plateaus and small gradients 7
4.3 Curvature changes the safe step 8

5 Momentum: remember consistent directions **9**
5.1 Practice: update momentum state 9
5.2 Nesterov look-ahead 10

6 Adaptive coordinate-wise scaling **11**
6.1 AdaGrad 11
6.2 RMSProp 11
6.3 Adam 12

7 Learning-rate schedules **13**

8 Read the training run **14**
8.1 Practice: optimization failure or overfitting? 15

9 A durable optimization routine **15**

PART III · OPTIMIZATION AND GENERALIZATION

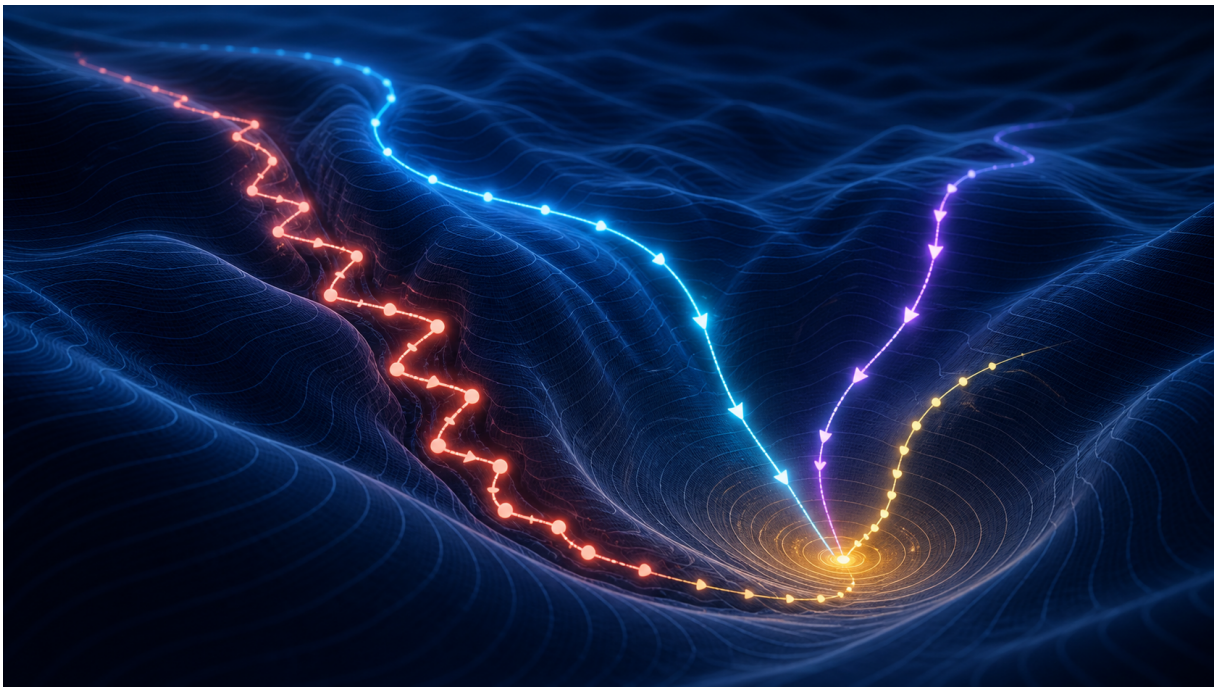


Figure 1: Several optimization trajectories descend through a contoured loss landscape toward a shared basin. Original course illustration generated for this chapter.

Backpropagation computes how the loss changes with every parameter. It does not decide what to do with that information. **Optimization is the rule that turns local gradient evidence into a trajectory through parameter space.**

By the end of this chapter, you should be able to

1. Interpret a parameter setting as a point on a loss landscape.
2. Separate the direction and step-size decisions in gradient descent.
3. Explain failures caused by ravines, plateaus, noise, and curvature.
4. Derive classical momentum and explain Nesterov look-ahead.
5. Compare AdaGrad, RMSProp, and Adam by the state they store.
6. Distinguish stochastic, minibatch, and full-batch gradient regimes.
7. Diagnose training runs from aligned loss, learning-rate, and gradient traces.

1 From a model to a landscape

Consider the same two-parameter binary classifier used in the lecture:

$$z = w_1 x_1 + w_2 x_2, \qquad \hat{y} = \sigma(z), \qquad \theta = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}.$$

Each choice of (w_1, w_2) defines a complete classifier. It produces predictions on the training set and therefore receives one loss:

$$(w_1, w_2) \longmapsto \mathcal{L}(w_1, w_2).$$

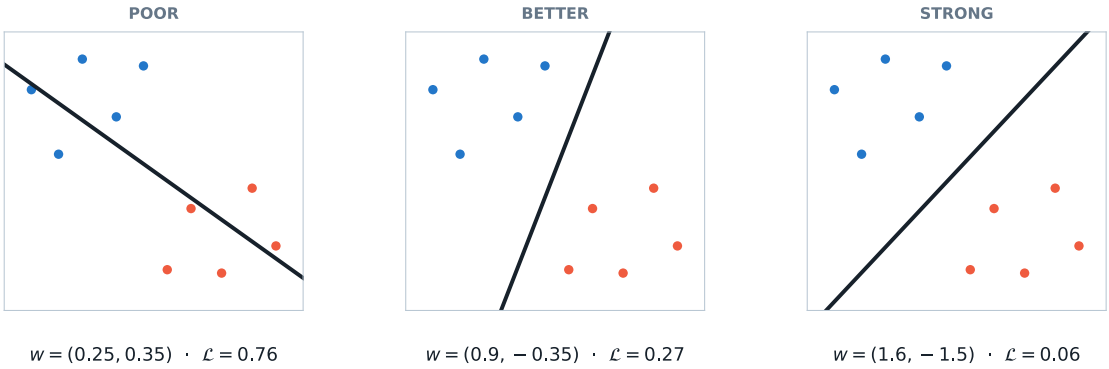


Figure 2: Three parameter settings produce three decision boundaries and three losses for the same data and model family.

A **loss landscape** is simply the loss evaluated across parameter settings. For two parameters we can draw it exactly as contours or height. A real network with P parameters lives in P dimensions; a published two-dimensional landscape is normally a slice through that space, not the complete object.

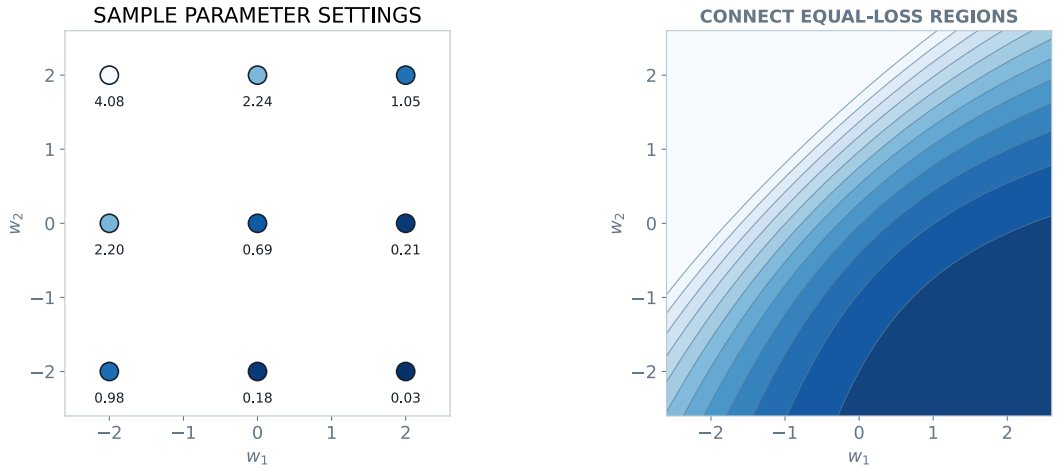


Figure 3: Sampled parameter settings, contours, and a surface are three representations of the same loss function.

The optimizer never sees the complete map. At step t, it has the current parameters, a loss estimate, and a local gradient:

$$\theta_t, \qquad \mathcal{L}(\theta_t), \qquad g_t = \nabla_\theta \mathcal{L}(\theta_t).$$

Its task is to choose the next point.

2 Gradient descent: direction and distance

For the scalar loss

$$\mathcal{L}(w) = (w-1)^2, \qquad w_t = 3,$$

the derivative is $\nabla_w \mathcal{L}(3) = 4$. A positive derivative says that increasing w raises the loss locally, so decreasing w lowers it.

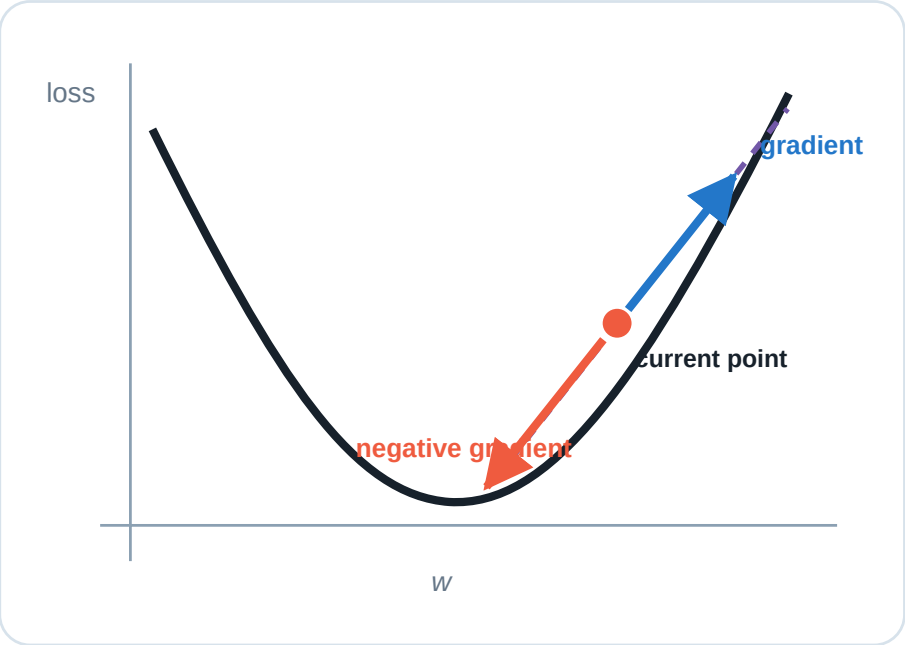


Figure 4: At the current point, the gradient follows the increasing tangent direction and the negative gradient points downhill.

In several dimensions, the gradient is the direction of steepest infinitesimal increase under the usual Euclidean geometry. Gradient descent moves in the opposite direction:

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t).} \tag{1}$$

This rule contains two decisions:

- $-\nabla_\theta \mathcal{L}$ chooses a **local direction**;
- the learning rate $\eta > 0$ chooses the **distance** traveled.

The word *local* matters. The gradient predicts improvement for an infinitesimal move. A finite step can cross a valley and increase the loss.

2.1 Practice: execute one update

Given

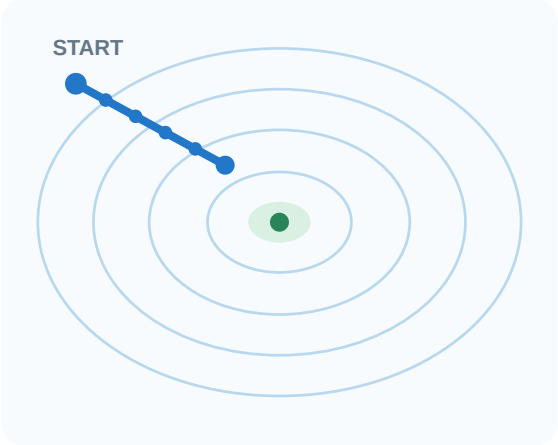
$$\theta_t = \begin{bmatrix} 0.25 \\ 0.35 \end{bmatrix}, \qquad g_t = \begin{bmatrix} 0.40 \\ -0.20 \end{bmatrix}, \qquad \eta = 0.5,$$

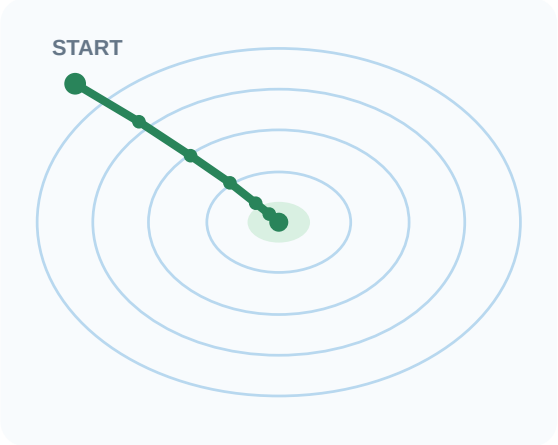
compute θ_{t+1} and state how each parameter moves.

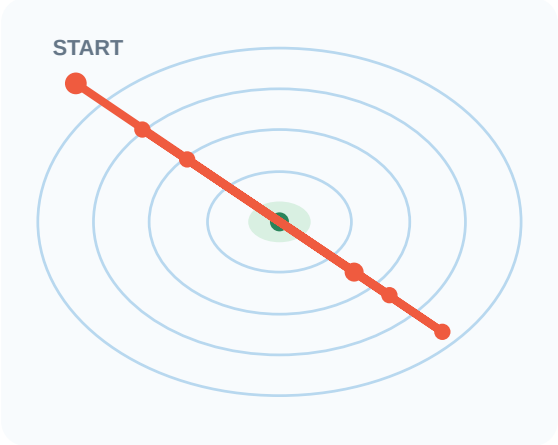
> **Worked solution**
>
> $\theta_{t+1} = [0.05, 0.45]^\top$. The first parameter decreases by 0.20 and the second increases by 0.10 because its gradient component is negative.

2.2 Learning-rate regimes

The following paths use the same landscape, starting point, and update rule. Only η changes.







Small is not *wrong*: it trades time for conservative movement. Large is not automatically *fast*: oscillation or divergence may prevent useful progress.

3 Three gradient regimes

Let a training set contain N examples and let B_t be the examples used at update t.

Regime	Batch size	Gradient	Main tradeoff
Stochastic gradient descent	$\lvert B_t \rvert = 1$	one-example estimate	very noisy, very frequent updates
Minibatch SGD	$1 < \lvert B_t \rvert < N$	subset estimate	efficient parallel work with moderate noise
Full-batch gradient descent	$\lvert B_t \rvert = N$	exact finite-dataset gradient	expensive updates, low sampling variation

In deep learning, **SGD usually means minibatch SGD**, even though the strict stochastic case uses one example. The word *batch* is ambiguous; say *minibatch* or *full batch*.

The minibatch gradient

$$g_t = \nabla_\theta \mathcal{L}_{B_t}(\theta_t)$$

is an estimate of the full-training-set gradient. Different minibatches produce different estimates at the same parameter point.

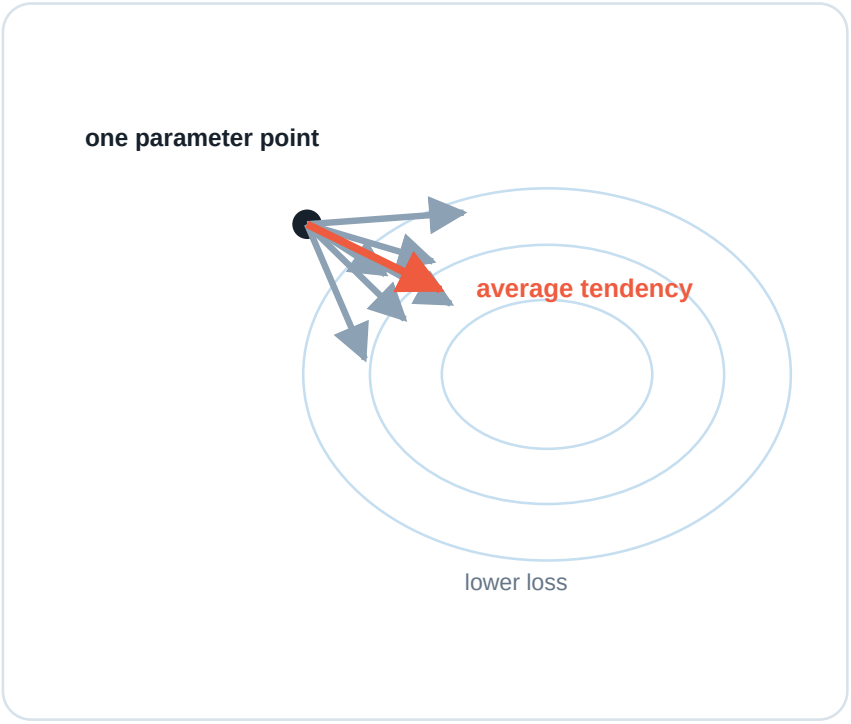


Figure 5: Several minibatches propose different negative-gradient update directions; their average follows the full-data tendency.

Larger minibatches usually reduce gradient variance, but they cost more computation per update and change hardware and generalization tradeoffs. “Exact dataset gradient” also does not mean the unknown population gradient is known.

4 Difficult terrain

4.1 Ravines and unequal curvature

A ravine is steep across one direction and shallow along another. The across-valley gradient dominates, flips sign on the opposite wall, and produces a zig-zag.

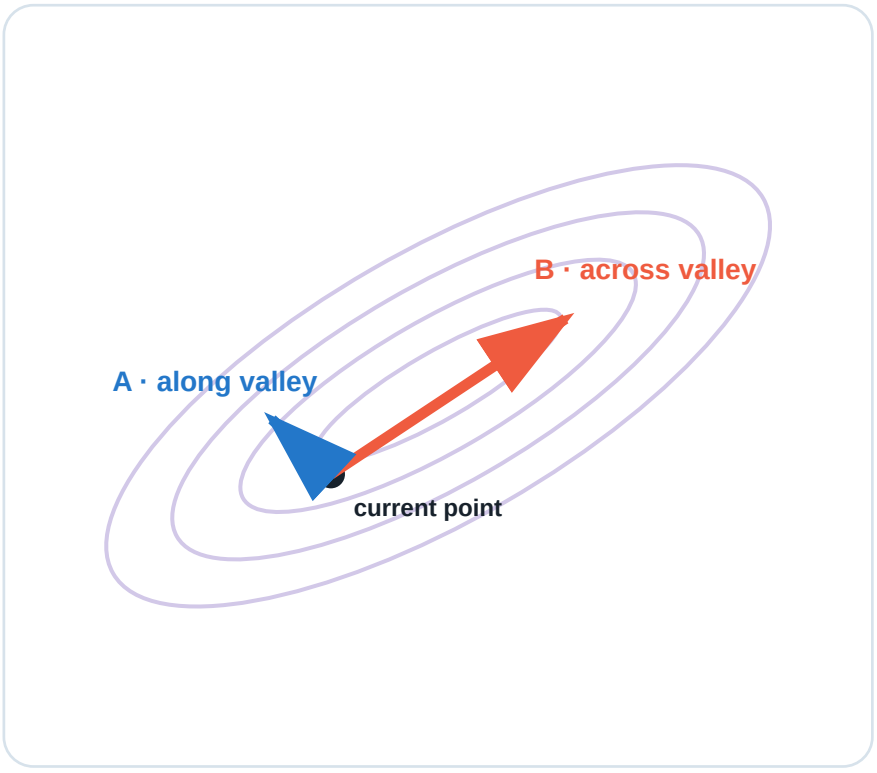


Figure 6: The long across-valley component dominates the small along-valley component.

Reducing η suppresses bouncing, but also slows progress along the shallow direction. One scalar learning rate must compromise between incompatible scales.

4.2 Plateaus and small gradients

A small gradient is not sufficient evidence that training has succeeded.

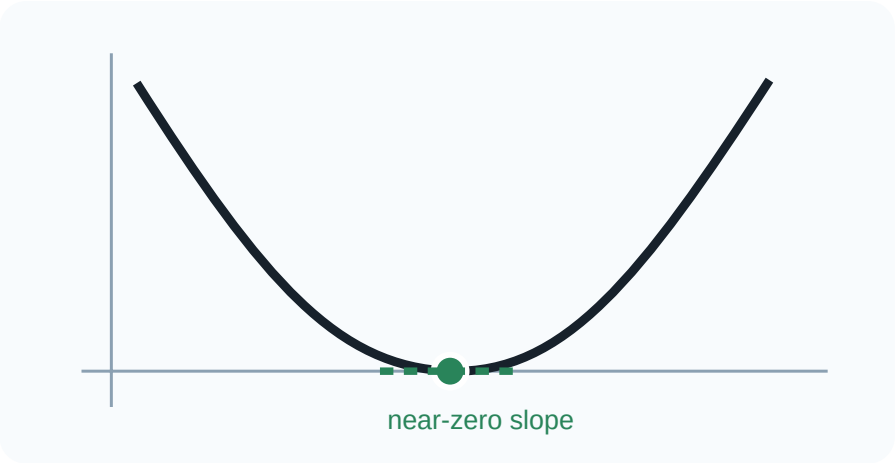


Figure 7: A near-zero gradient can occur at a low minimum.

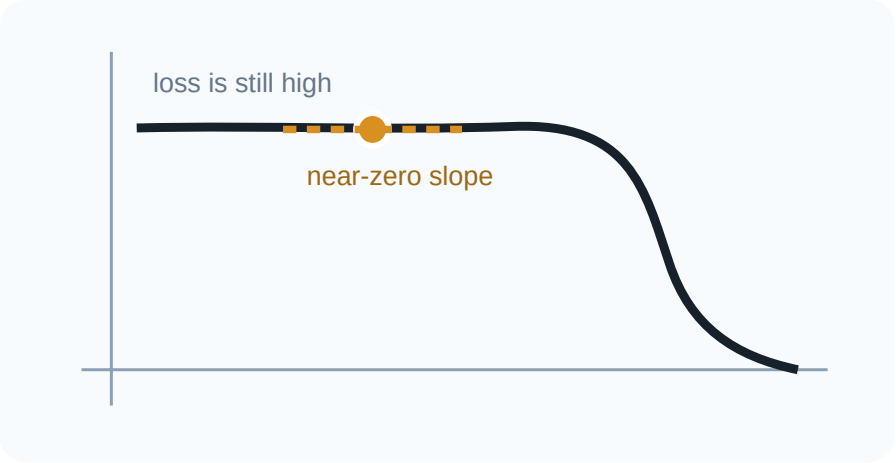
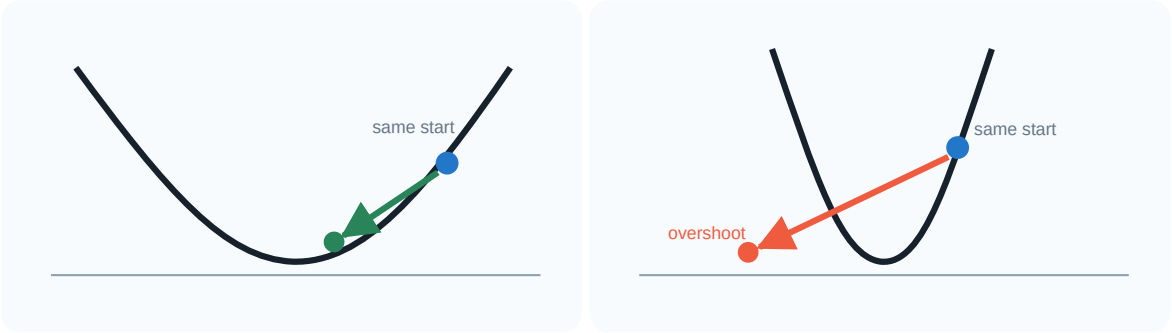


Figure 8: A near-zero gradient can also occur on a high plateau.

Use the loss level and recent trajectory together with $\|g_t\|$. In high-dimensional networks, saddle-like regions provide another reason for weak local gradients.

4.3 Curvature changes the safe step

The same learning rate can be safe on a gentle quadratic and overshoot on a sharper one.



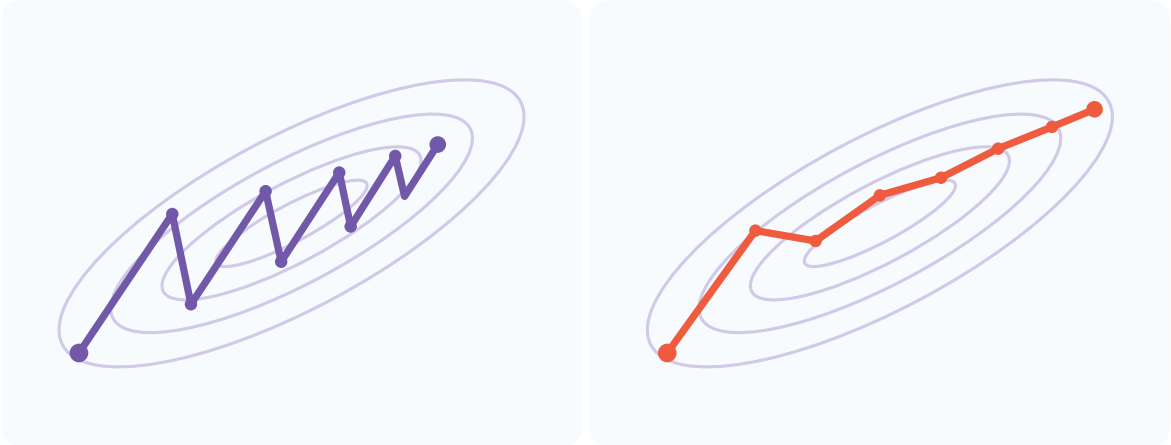
The practical conclusion is not that curvature must be computed exactly. It is that a useful step scale can differ across directions and change during training.

5 Momentum: remember consistent directions

Plain gradient descent is memoryless. Classical momentum stores a running gradient direction:

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}, \qquad 0 \le \beta < 1. \tag{2}$$

Consistent components reinforce; alternating components cancel. This is why momentum can damp ravine oscillation while accumulating progress along the valley.



Larger β preserves history longer. This smooths noise and strengthens persistence, but makes the optimizer slower to respond when the useful direction changes. Some references use

$$v_{t+1} = \beta v_t + (1-\beta) g_t.$$

The stored variable then has a different scale; the memory idea is unchanged.

5.1 Practice: update momentum state

Let

$$\theta_t = \begin{bmatrix} 1.00 \\ -0.50 \end{bmatrix}, \quad v_t = \begin{bmatrix} 0.60 \\ -0.20 \end{bmatrix}, \quad g_t = \begin{bmatrix} 0.20 \\ 0.40 \end{bmatrix}, \quad \beta = 0.9, \quad \eta = 0.1.$$

Compute v_{t+1} and θ_{t+1}.

Worked solution

$v_{t+1} = [0.74, 0.22]^\top$, followed by $\theta_{t+1} = [0.926, -0.522]^\top$.

5.2 Nesterov look-ahead

Nesterov momentum evaluates the new gradient where stored momentum is about to take the parameters:

$$\theta_t^{\text{look}} = \theta_t - \eta\beta v_t,$$

$$v_{t+1} = \beta v_t + \nabla \mathcal{L}(\theta_t^{\text{look}}), \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}.$$

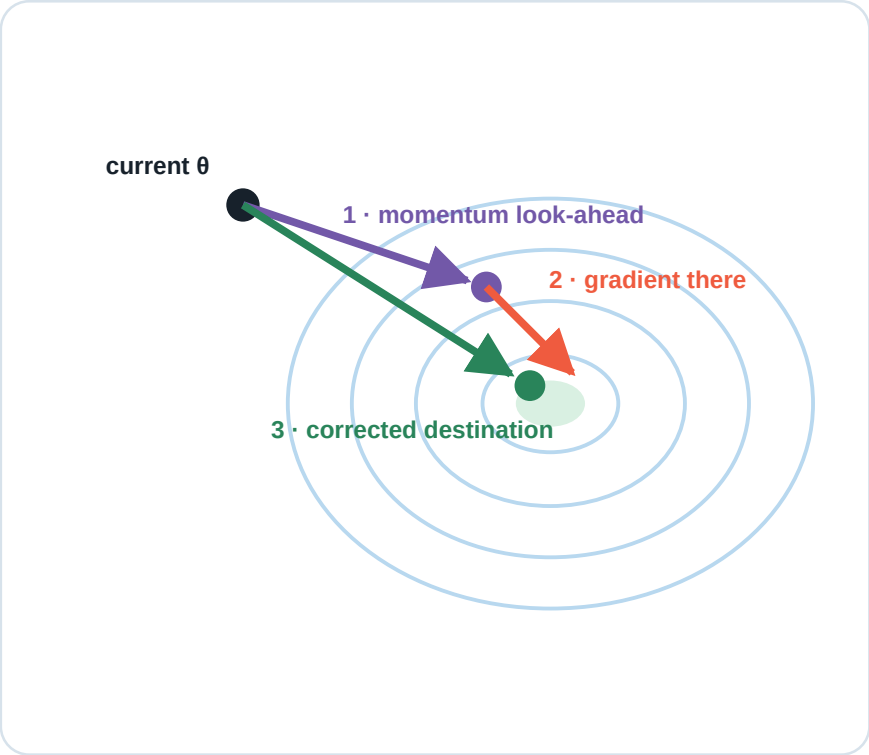


Figure 9: Nesterov follows momentum to a provisional point, measures the gradient there, and forms a corrected destination.

Equivalent formulas appear because implementations store and scale velocity differently. The conceptual distinction is stable: **look where momentum is taking you, then correct using evidence there**.

6 Adaptive coordinate-wise scaling

Momentum remembers direction, but a global η still scales every coordinate equally. Adaptive methods store a separate magnitude estimate for every parameter.

6.1 AdaGrad

AdaGrad accumulates squared gradients (Duchi et al. 2011):

$$s_t = s_{t-1} + g_t \odot g_t, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \varepsilon} \odot g_t.$$

All operations are element-wise. Squaring removes sign because s_t measures scale, not direction. Repeatedly large coordinates develop larger denominators and smaller effective steps.

AdaGrad's strength is also its limitation: s_t can only increase, so $\eta/\sqrt{s_t}$ can only decrease. Early large gradients can permanently suppress later updates.

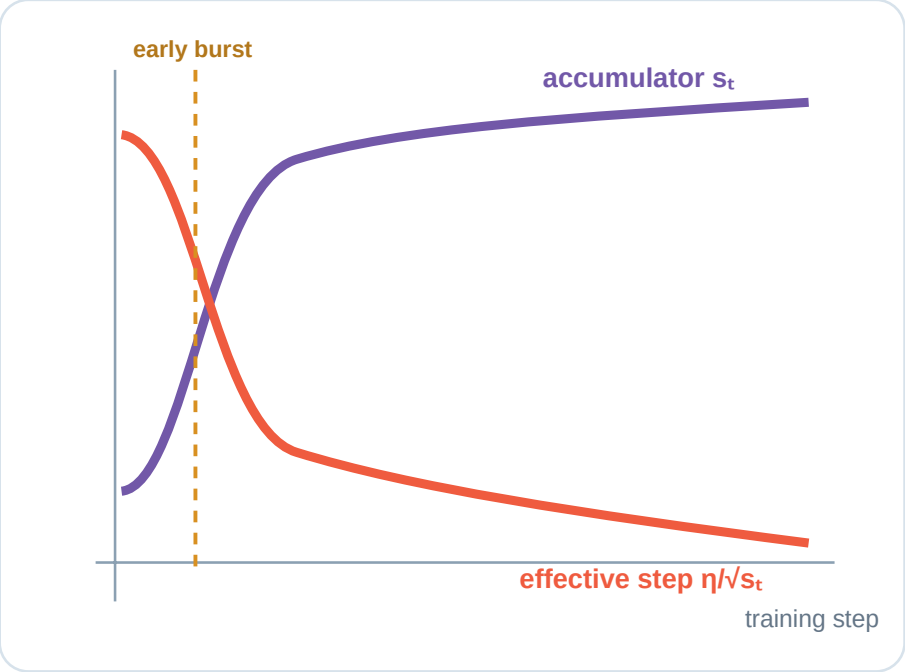


Figure 10: AdaGrad's accumulator rises while its effective learning rate shrinks after an early gradient burst.

6.2 RMSProp

RMSProp replaces the permanent sum with an exponentially weighted moving average (Tieleman and Hinton 2012):

$$s_t = \rho s_{t-1} + (1-\rho) g_t \odot g_t, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \varepsilon} \odot g_t.$$

Old squared gradients fade, allowing the coordinate scale to adapt when the recent regime changes.

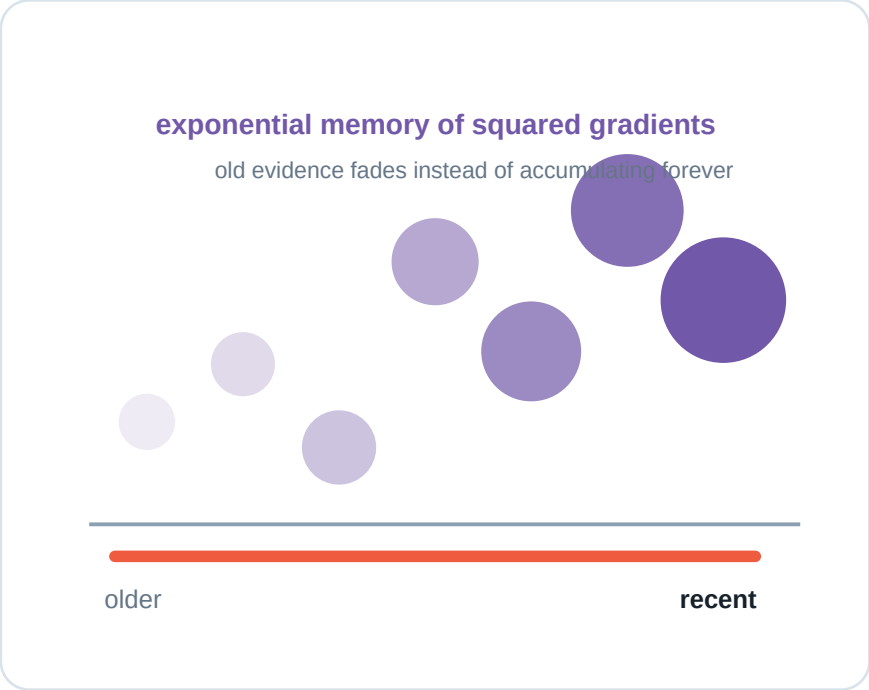


Figure 11: RMSProp emphasizes recent squared gradients while older observations fade.

6.3 Adam

Adam combines two exponential memories (Kingma and Ba 2015):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

Here m_t stores direction and v_t stores squared-gradient scale. The symbol v is overloaded: it is **not** the classical-momentum velocity used earlier. Because both states start at zero, Adam applies bias corrections

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

then updates

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Optimizer	Direction memory	Scale memory	Main tradeoff
Gradient descent	none	none	simple, one global scale
Momentum	yes	none	steadier path, can overshoot turns
AdaGrad	none	all past squares	effective steps may decay too far
RMSProp	none	recent squares	adaptive scale without direction memory
Adam	yes	recent squares	convenient strong baseline with more state

This table organizes information, not a universal ranking. Adam still requires a suitable learning rate and careful evaluation.

7 Learning-rate schedules

A schedule changes the global learning rate over training while the optimizer defines how the current gradient and stored state become an update.



Figure 12: Constant, step-decay, cosine-decay, and warmup-plus-decay schedules.

- **Constant:** useful for a controlled baseline.
- **Step decay:** reduces the rate at chosen milestones.
- **Cosine decay:** decreases smoothly toward a small final rate.
- **Warmup plus decay:** approaches the target rate gradually when early updates are fragile, then refines with decay.

No schedule is automatically best. The recurring rationale is coarse early movement followed by finer late adjustment.

8 Read the training run

Training produces aligned traces. Interpret them together:

- training loss: is optimization improving its objective?
- validation loss: does improvement generalize?
- learning rate: what global step scale was applied?
- gradient norm: was the local signal weak, ordinary, or explosive?



Figure 13: Training loss, validation loss, learning rate, and gradient norm aligned over epochs.

Visible evidence	First hypothesis	Next controlled test
Loss decreases painfully slowly	step scale may be too small	increase η modestly; hold other factors fixed
Loss oscillates or becomes non-finite	steps or gradients may be too large	lower η; inspect gradient norm and numerics
Loss is high and gradients are near zero	plateau, saturation, or dead path	inspect activations and initialization
Training improves while validation worsens	overfitting	retain best checkpoint; test regularization

8.1 Practice: optimization failure or overfitting?

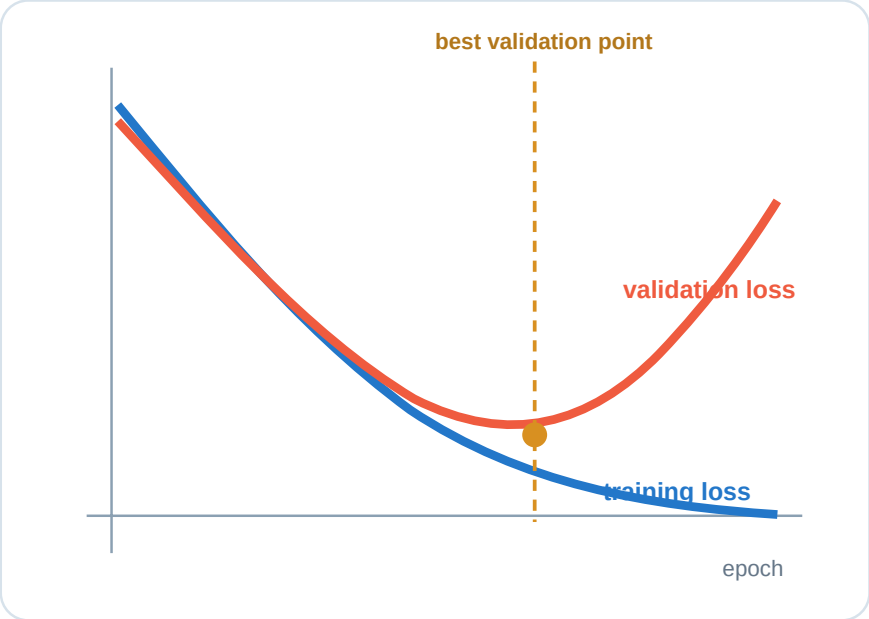


Figure 14: Training loss continues downward while validation loss turns upward after its best point.

Choose the best diagnosis: learning rate too small, learning rate too large, or overfitting.

> **Diagnosis**
>
> The primary symptom is overfitting: training loss falls while validation loss rises. Retain the best validation checkpoint and test a generalization intervention.

9 A durable optimization routine

1. Establish a simple baseline with recorded hyperparameters and seeds.
2. Inspect aligned training, validation, learning-rate, and gradient traces.

3. Form a hypothesis from visible evidence.
4. Change one factor and rerun under comparable conditions.
5. Judge the result with held-out evaluation, not training loss alone.

Optimization sits inside the five-part learning frame:

$$\text{data} \rightarrow \text{hypothesis} \rightarrow \text{loss} \rightarrow \boxed{\text{optimization}} \rightarrow \text{evaluation}.$$

An optimizer is not a magic model chooser. It is a rule for turning local evidence into a training trajectory.

Duchi, John, Elad Hazan, and Yoram Singer. 2011. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.” *Journal of Machine Learning Research* 12: 2121–59. https://jmlr.org/papers/v12/duchi11a.html.

Kingma, Diederik P., and Jimmy Ba. 2015. “Adam: A Method for Stochastic Optimization.” *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980.

Tieleman, Tijmen, and Geoffrey Hinton. 2012. *Lecture 6.5—RMSProp: Divide the Gradient by a Running Average of Its Recent Magnitude*. COURSERA: Neural Networks for Machine Learning. https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.